

Licence L3 Informatique - Université Paris Cité
Année universitaire 2024/2025

Rapport projet



Chatbot pour la recommandation des smartphones

Unité d'enseignement : Linguistique

Étudiante : Berrais Wafaa

Sommaire

Introduction.....	3
Technologies de l'Intelligence Conversationnelle et Génération Augmentée	4..6
2.1. Traitement Automatique du Langage Naturel (NLP)	
2.2. Agents de Dialogue et Systèmes Conversationnels	
2.3. Modèles de Langage (LLMs)	
2.4. Architecture de Génération Augmentée par Récupération (RAG)	
Schéma représentatif de système RAG	
Données et Prétraitement.....	6..9
3.1. Description du Jeu de Données	
3.2. Prétraitement des Données	
3.2.1. Sélection des Caractéristiques Pertinentes	
3.2.2. Gestion des Valeurs Manquantes	
3.3. Vectorisation et Indexation Sémantique des Avis	
3.4. Analyse Sentimentale et Filtrage des Résultats	
Conception et Implémentation du Système.....	9..13
Schéma résumant notre système ChatPhone	
4.1. Outils et Technologies Clés	
4.2. API de Récupération d'Informations	
4.3. Backend Orchestrateur et Module de Génération	
4.5. Interface Utilisateur (Frontend)	
Discussion et perspectives.....	14
Conclusion.....	15
Références.....	16

1. Introduction

L'acquisition d'un nouveau smartphone représente une décision de plus en plus complexe pour le consommateur, confronté à une offre pléthorique et à une sophistication technique croissante. Si les systèmes d'aide à la décision traditionnels et les FAQ offrent un premier niveau d'information, ils peinent souvent à appréhender la subtilité des requêtes formulées en langage naturel et à fournir des conseils véritablement personnalisés. L'avènement des grands modèles de langage (LLMs) a ouvert des perspectives prometteuses pour des interactions plus intuitives. Néanmoins, leur utilisation directe dans un contexte commercial spécifique, tel que la recommandation de produits pour une plateforme comme Amazon, soulève un défi majeur, un LLM non contraint pourrait aisément suggérer des articles non référencés, obsolètes, voire inexistants, compromettant ainsi l'expérience utilisateur et l'efficacité commerciale.

Ce projet s'attaque à cette problématique en proposant la conception d'un chatbot intelligent, capable de fournir des recommandations de smartphones pertinentes et fiables, ancrées exclusivement dans le catalogue de produits d'Amazon. Pour ce faire, nous avons opté pour une architecture RAG (Retrieval Augmented Generation). Cette approche stratégique permet de conjuguer la capacité de compréhension et de génération de langage naturel d'un LLM ici, le modèle Ollama Qwen avec un mécanisme de récupération d'informations préalable. Concrètement, avant de formuler une réponse, le système consulte la base de données des smartphones Amazon pour en extraire les données les plus pertinentes. Ce processus garantit que les recommandations sont non seulement contextuellement adaptées à la requête de l'utilisateur, mais aussi strictement alignées sur l'inventaire réel de la plateforme, évitant ainsi les suggestions inappropriées.

Ce rapport se propose de détailler la méthodologie adoptée pour la réalisation de ce chatbot, depuis les fondements théoriques sous-jacents jusqu'à l'implémentation technique et l'interface utilisateur. Nous exposerons le traitement des données issues d'Amazon, l'architecture du système RAG, et les choix technologiques opérés. L'objectif est de démontrer comment cette approche permet de créer un outil d'aide à la décision à la fois performant, fiable et centré sur les besoins de l'utilisateur dans un contexte d'achat spécifique.

2. Technologies de l'Intelligence Conversationnelle et Génération Augmentée

Cette section présente les technologies d'IA fondamentales de notre chatbot le NLP pour l'interprétation du langage, les LLMs pour la compréhension et la génération, et l'architecture RAG pour des réponses intelligentes basées sur des données spécifiques.

2.1. Traitement Automatique du Langage Naturel (NLP)

Le Traitement Automatique du Langage Naturel (NLP), un sous-domaine de l'IA, permet aux ordinateurs de comprendre et communiquer en langage humain grâce au machine learning et à l'apprentissage profond. Essentiel à l'IA générative et aux LLMs, le NLP est omniprésent, des moteurs de recherche aux assistants virtuels. Pour notre chatbot, le NLP est important. Il intervient via la tokenisation, qui segmente les requêtes et descriptions en unités textuelles. Ensuite, la vectorisation transforme ces unités en vecteurs numériques, capturant leur sémantique. Ce processus permet de mesurer la similarité entre la demande de l'utilisateur et les données des smartphones, une étape indispensable pour le module de récupération de notre système RAG [1].

2.2. Agents de Dialogue et Systèmes Conversationnels

Les agents de dialogue (ou chatbots) sont des programmes simulant une conversation humaine pour comprendre les intentions des utilisateurs et fournir des réponses adaptées, que ce soit pour informer, accomplir des tâches ou assister. Initialement basés sur des règles, ils ont évolué grâce au NLP et au machine learning vers des systèmes plus sophistiqués, incluant des approches basées sur la récupération ou la génération (souvent via des LLMs), et peuvent être orientés tâche ou conversation ouverte. Notre chatbot de recommandation de smartphones est un agent de dialogue orienté tâche, visant à guider l'utilisateur dans son choix en s'appuyant sur le NLP, les LLMs et l'architecture RAG pour des réponses pertinentes et contextuelles.

2.3. Modèles de Langage (LLMs)

Les Modèles de Langage à grande échelle (LLMs), tels que Qwen utilisé dans ce projet, sont des IA avancées du NLP, entraînées sur d'immenses corpus textuels pour maîtriser les

subtilités du langage humain. Leur fonctionnement s'appuie fréquemment sur l'architecture "Transformer", qui leur permet, via un pré-entraînement (par exemple, prédire le mot suivant), d'acquérir une compréhension approfondie de la syntaxe, de la sémantique et des connaissances générales encodées dans les données. Ces modèles excellent dans la génération de texte naturel et la compréhension de requêtes complexes. Néanmoins, ils ont des limites notables, leur base de connaissances est statique (limitée à leurs données d'entraînement) et ils peuvent occasionnellement "halluciner" c'est-à-dire produire des informations incorrectes avec assurance ou reproduire des biais présents dans leurs données d'apprentissage.

2.4. Architecture de Génération Augmentée par Récupération (RAG)

L'architecture RAG (Retrieval Augmented Generation) vise à surmonter les limites des LLMs, comme leur manque de connaissances spécifiques ou à jour et leur propension aux hallucinations. Elle combine un LLM avec un système de récupération d'informations externe : avant de générer une réponse, le LLM est "augmenté" par des données pertinentes extraites d'une base de connaissances[2]. Cette approche offre des avantages majeurs les parmi : les réponses sont fondées sur des faits spécifiques et récents, réduisant les hallucinations et augmentant la fiabilité. Elle améliore aussi la transparence (sources traçables) et permet de mettre à jour les connaissances sans ré-entraîner le LLM. Un système RAG comprend typiquement :

- Le Retriever : Il recherche et extrait les informations pertinentes d'une base de données externe (ici, les produits Amazon) en réponse à la requête de l'utilisateur, souvent via une recherche de similarité vectorielle.
- Le Generator (un LLM) : Il utilise la requête initiale et les informations récupérées pour formuler la réponse finale en langage naturel.

L'utilisation de RAG est justifiée pour notre chatbot de recommandation de smartphones.

Elle garantit que les conseils sont basés sur les données réelles du catalogue Amazon, évitant que le LLM (Ollama Qwen) ne propose des produits existants ou des informations erronées. Le Retriever fournit les données factuelles, et le Generator les synthétise en une réponse utile et conversationnelle.

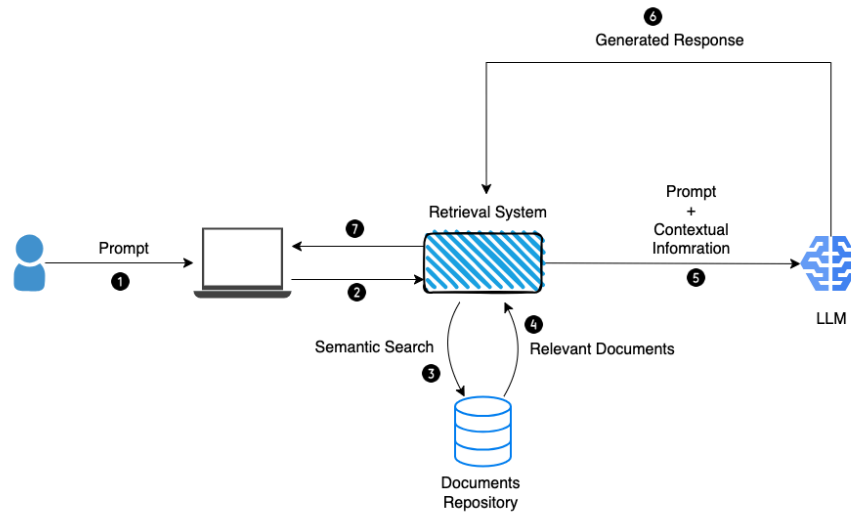


Schéma représentatif de système RAG [2]

3. Données et Prétraitement

La qualité et la pertinence des données sont fondamentales pour le bon fonctionnement d'un système RAG. Cette section décrit le jeu de données utilisé pour alimenter notre chatbot, les étapes de prétraitement appliquées pour le nettoyer et le structurer, ainsi que l'intégration de l'analyse de sentiment pour affiner la pertinence des informations récupérées.

3.1. Description du Jeu de Données

Notre projet s'appuie sur un jeu de données public portant sur les téléphones portables et leurs avis, disponible sur la plateforme Kaggle : "Amazon Cell Phones Reviews" (accessible via [3]). Ce dataset est initialement composé de deux fichiers CSV distincts :

- 20190928-items.csv : Ce fichier contient des informations descriptives sur les produits (téléphones portables). Les attributs principaux incluent :
 1. asin : L'identifiant unique du produit sur Amazon.
 2. brand : La marque du téléphone.
 3. title : Le nom ou la description du produit.
 4. url : L'URL de la page du produit.
 5. image : L'URL de l'image du produit.

6. rating : La note moyenne globale du produit.
 7. reviewUrl : L'URL de la page des avis du produit.
 8. totalReviews : Le nombre total d'avis pour le produit.
 9. prices : Le prix du produit.
- 20190928-reviews.csv : Ce fichier regroupe les avis laissés par les utilisateurs pour les produits. Les attributs notables sont :
 1. asin : L'identifiant du produit auquel l'avis se réfère.
 2. name : Le nom de l'auteur de l'avis.
 3. rating : La note donnée par l'utilisateur dans son avis (de 1 à 5).
 4. date : La date de publication de l'avis.
 5. verified : Un booléen indiquant si l'achat a été vérifié.
 6. title : Le titre de l'avis.
 7. body : Le contenu textuel complet de l'avis.
 8. helpfulVotes : Le nombre de votes utiles reçus par l'avis.

Ces deux fichiers ont été fusionnés en un seul DataFrame Pandas en utilisant l'identifiant asin comme clé de jointure “pd.merge(item, review, how='right', on='asin')”. Le DataFrame résultant combine ainsi les détails de chaque produit avec chaque avis individuel le concernant.

3.2. Prétraitement des Données

Un prétraitement rigoureux a été appliqué pour préparer les données à la fois pour l'analyse et pour l'alimentation du système de récupération d'information.

3.2.1. Sélection des Caractéristiques Pertinentes

Après la fusion, une sélection des colonnes a été effectuée pour ne conserver que les attributs jugés les plus pertinents pour notre tâche de recommandation et d'analyse textuelle. Les colonnes conservées sont : asin, brand, title_x, rating_x, totalReviews, prices, rating_y, verified, title_y, et body.

Justification des choix :

- Les attributs url, image, reviewUrl ont été écartés car ils ne sont pas directement exploitables pour une analyse textuelle et un RAG basé sur du texte.
- L'attribut name a été supprimé pour des raisons de confidentialité et parce qu'il n'apporte pas d'information pertinente sur les caractéristiques du produit.
- L'attribut date pourrait être utile pour analyser l'évolution des avis ou filtrer par récence, mais pour simplifier la complexité initiale, il n'a pas été retenu comme critère principal de récupération.

- `helpfulVotes` pourrait servir d'indicateur de la qualité ou de la pertinence d'un avis, mais nous nous sommes concentrés sur le contenu textuel et la note de l'avis (`rating_y`) ainsi que le statut `verified` pour évaluer la fiabilité.
- Les attributs conservés (`brand`, `title_x`, `prices`, `rating_x`, `totalReviews`) fournissent un contexte essentiel sur le produit, tandis que `title_y`, `body`, `rating_y` et `verified` sont cruciaux pour comprendre le contenu et la valeur de chaque avis individuel.

3.2.2. Gestion des Valeurs Manquantes

L'analyse des valeurs manquantes (`df.isnull().sum()`) a révélé des absences principalement dans la colonne `prices`, et de manière moins significative dans `title_y` et `body`. Pour garantir la qualité des données utilisées par le modèle, toutes les lignes contenant au moins une valeur manquante dans les colonnes sélectionnées ont été supprimées en utilisant `df.dropna(inplace=True)`.

Justification des choix :

- La présence d'un prix (`prices`) est considérée comme une information essentielle pour une recommandation pertinente.
- Le titre (`title_y`) et surtout le corps (`body`) d'un avis sont indispensables pour l'analyse de sentiment et la vectorisation. Un avis sans contenu textuel n'a aucune valeur pour notre système RAG.
- Bien que la suppression de lignes puisse réduire la taille globale du jeu de données, elle assure que chaque entrée traitée par la suite est complète et exploitable, améliorant ainsi la fiabilité des étapes ultérieures.

3.3. Vectorisation et Indexation Sémantique des Avis

Pour permettre une recherche sémantique efficace au cœur de notre système RAG, les données textuelles des avis ont subi un processus de transformation en représentations vectorielles (`embeddings`) et ont été indexées.

La première étape a consisté à préparer le corpus textuel. Afin de fournir un contexte riche pour chaque avis, une colonne `full_text` a été créée, agrégeant des informations clés telles que la marque, le prix, le titre du produit, ainsi que le titre et le corps de l'avis. Cette chaîne de caractères descriptive et unifiée pour chaque entrée a ensuite servi de base à la vectorisation. Ce processus de transformation du texte en vecteurs numériques a été réalisé à l'aide du modèle `SentenceTransformer('all-MiniLM-L6-v2')`. Ce modèle, fondé sur l'architecture Sentence-BERT, est reconnu pour sa capacité à produire des `embeddings` de phrases de haute qualité, optimisés pour les tâches de similarité sémantique, tout en présentant un bon compromis entre performance et taille. Chaque entrée de la colonne

full_text a ainsi été encodée en un vecteur d'embedding dense, capturant sa signification sémantique.

Ces embeddings ont ensuite été utilisés pour construire un index de recherche performant avec la bibliothèque FAISS, spécifiquement un `faiss.IndexFlatL2`. FAISS est une solution optimisée pour la recherche de similarité dans de grands ensembles de vecteurs. L'index `IndexFlatL2` effectue une recherche exhaustive basée sur la distance Euclidienne, garantissant la récupération des voisins les plus proches exacts, une approche adaptée à la volumétrie de notre jeu de données. Une fonction de recherche sémantique a été implémentée, elle vectorise la requête de l'utilisateur avec le même modèle `all-MiniLM-L6-v2` et interroge cet index FAISS pour retourner les top_k avis les plus sémantiquement pertinents, formant ainsi la base du module de récupération de notre système RAG.

3.4. Analyse Sentimentale et Filtrage des Résultats

Afin d'affiner la pertinence des résultats et de mieux s'aligner sur l'intention de l'utilisateur, une analyse de sentiment a été intégrée au processus de récupération. L'objectif principal est d'éviter de fournir des avis sémantiquement pertinents mais exprimant un sentiment négatif lorsque l'utilisateur recherche des aspects positifs (par exemple, pour une "bonne caméra", privilégier les avis positifs mentionnant la caméra). Cela permet de fournir un contexte plus ciblé au module de génération du RAG.

Pour ce faire, l'analyse de sentiment a été réalisée avec la bibliothèque VADER, choisie pour son efficacité sur les textes d'avis et sa capacité à fournir un score de sentiment composite (compound score) sans entraînement préalable. Ce score a été calculé pour chaque avis. Ensuite, pour un filtrage efficace, deux sous-index FAISS distincts ont été créés : l'un pour les avis positifs (compound > 0.05) et l'autre pour les avis négatifs (compound < -0.05). La fonction de recherche sémantique a été adaptée pour interroger l'index approprié (positif, négatif, ou global) selon le sentiment désiré, assurant ainsi que le contexte fourni au LLM est plus en phase avec les attentes de l'utilisateur.

4. Conception et Implémentation du Système

Dans cette section, nous expliquons concrètement comment notre chatbot de recommandation de smartphones a été conçu et réalisé. Nous passerons en revue son architecture, l'implémentation de ses modules, et les technologies qui permettent de transformer une question utilisateur en une réponse utile.

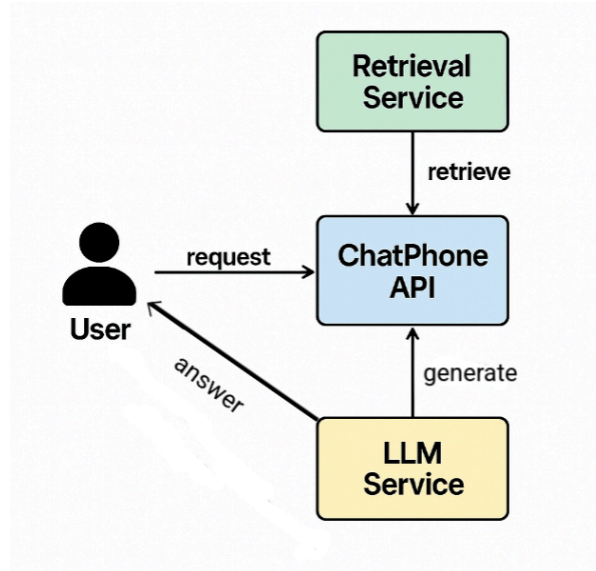


Schéma resumant notre systeme ChatPhone

4.1. Outils et Technologies Clés

La réalisation de ce projet s'est appuyée sur un ensemble d'outils et de bibliothèques Python, ainsi que sur des technologies web standard :

Langage et Manipulation de Données : Python comme langage principal, avec Pandas pour la manipulation des DataFrames.

Traitement du Langage Naturel et Recherche :

- sentence-transformers (avec le modèle all-MiniLM-L6-v2) pour la vectorisation sémantique des textes.
- faiss pour la création d'index et la recherche de similarité vectorielle efficace.

Développement Backend et API :

- FastAPI pour la création de l'API de récupération d'informations, servi par Uvicorn.
- Pydantic pour la validation des données des requêtes API.
- Flask pour le développement du serveur backend principal (orchestrateur).
- Flask-CORS pour la gestion des requêtes cross-origin.
- requests pour la communication HTTP entre le backend Flask et l'API FastAPI.
- pyngrok pour exposer l'API FastAPI locale sur une URL publique.

Modèle de Langage et Génération :

- Ollama pour l'exécution locale du modèle de langage qwen3:0.6b.
- Le module subprocess de Python pour interagir avec l'interface en ligne de commande d'Ollama.

Développement Frontend :

- HTML, CSS, et JavaScript pour la création de l'interface utilisateur web.

4.2. API de Récupération d'Informations

Pour gérer la recherche d'informations pertinentes dans notre base de données d'avis, une API dédiée a été développée avec FastAPI. Cette approche permet de découpler la logique de recherche du reste de l'application.

Rôle et Fonctionnalités : L'API expose un endpoint POST /search. Lorsqu'il est interrogé avec une requête textuelle et un nombre k de résultats souhaités, il vectorise la requête en utilisant le modèle all-MiniLM-L6-v2, interroge l'index FAISS pré-chargé (faiss_index_all.idx) pour trouver les k avis les plus similaires, et retourne ces informations (marque, modèle, corps de l'avis) sous forme de JSON.

Exposition : pyngrok est utilisé pour créer un tunnel sécurisé, rendant cette API locale accessible via une URL publique, ce qui permet au backend Flask de la consommer.

4.3. Backend Orchestrateur et Module de Génération

Le cœur de l'application repose sur un serveur Flask (app.py) qui se lance en acceptant l'URL de l'API de récupération soit en argument de ligne de commande, soit via la variable d'environnement NGROK_URL. Si cette URL n'est pas fournie, le serveur s'interrompt avec un message d'erreur clair. Une fois démarré, il expose un endpoint POST/chat qui reçoit la requête de l'utilisateur et, avant tout traitement, détecte si c'est une salutation ou un remerciement pour y répondre immédiatement.

Pour toute autre requête liée aux smartphones, le backend engage la fonction retrieve(query, k=20), laquelle envoie une requête HTTP à l'API Ngrok pour extraire jusqu'à vingt candidats triés par pertinence. Les cinq premiers extraits d'avis sont formatés en contexte textuel numéroté. Ce contexte est ensuite injecté dans un prompt unique, où l'on précise le rôle de ChatPhone, la liste des extraits, la question brute et les instructions impératives de formatage et de refus.

Le prompt issu de cette étape est transmis localement au modèle de langage qwen3:0.6b via Ollama, appelé en sous-processus (subprocess.Popen(["ollama", "run",

OLLAMA_MODEL, prompt])). La réponse brute est alors récupérée, nettoyée (suppression des espaces superflus et des balises internes), puis renvoyée au frontend sous forme de JSON. Cette orchestration garantit une séparation claire entre la récupération sémantique (RAG) et la génération (LLM) tout en offrant une configuration flexible du tunnel Ngrok et un contrôle strict du format de réponse.

4.5. Interface Utilisateur (Frontend)

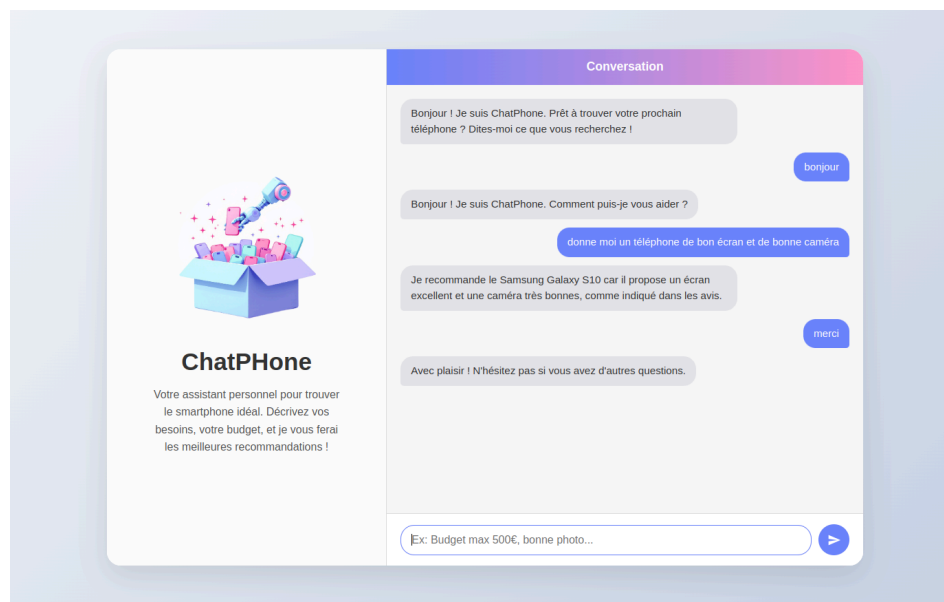
L'interaction avec l'utilisateur se fait via une interface web simple et intuitive, développée en HTML, CSS, et JavaScript (CHATPHONEtemplate.html).

Rôle et Fonctionnalités :

Elle présente une zone de saisie où l'utilisateur peut taper sa question.

- Les messages échangés (utilisateur et chatbot) sont affichés dans une zone de conversation.
- Le JavaScript gère l'envoi asynchrone de la requête de l'utilisateur au backend Flask (/chat).
- Un message temporaire "Je traite votre demande..." est affiché pendant que le backend et le LLM travaillent.
- La réponse finale du chatbot est ensuite affichée. Le code JavaScript inclut également une étape de nettoyage pour supprimer les éventuelles balises

Exemple de discussion :

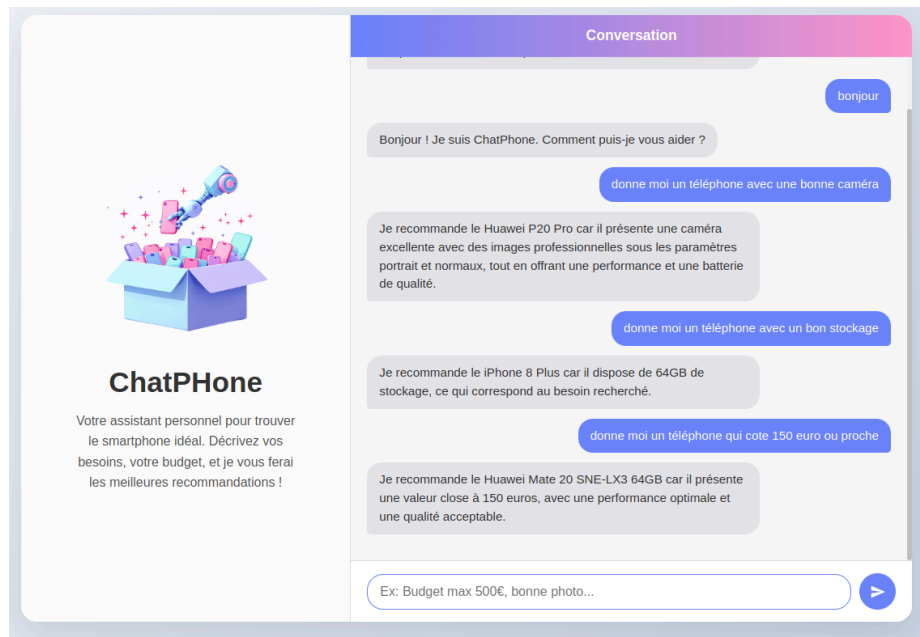


Interface graphique de ChatPHone

Cas de testes :

Cas de test 1 : Requêtes « normales »

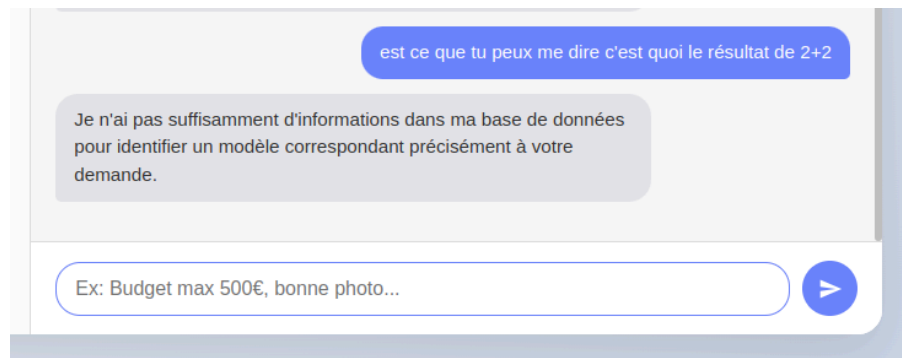
- Objectif : Vérifier que ChatPhone envoie bien une recommandation pour une demande de smartphone.
- Requête : « Donne-moi un téléphone avec une bonne caméra »
- Résultat



Resultat de cas de test 1

Cas de test 2 Requête hors domaine

- Objectif : Vérifier que ChatPhone refuse poliment toute question qui ne concerne pas les smartphones.
- Requête : « Quelle est la racine carrée de 81 ? »
- Résultat



Resultat cas de test 2

5. Discussion et perspectives

Notre chatbot ChatPhone démontre une capacité convaincante à comprendre des requêtes variées sur les smartphones qu'il s'agisse de critères techniques comme "une bonne caméra", "256 GB de stockage" ou "4 GB de RAM" et à extraire des extraits d'avis pertinents pour formuler une recommandation concise ou, le cas échéant, un message de refus pour les questions hors domaine. Grâce à une architecture RAG reposant sur FAISS et des embeddings SentenceTransformer, il parvient à agréger un contexte sémantique adapté et à générer une réponse au format strict défini par nos instructions.

Cependant, plusieurs limitations demeurent. La pertinence des recommandations dépend fortement de la présence d'extraits explicites dans le contexte : dès lors qu'aucun avis ne mentionne clairement la caractéristique recherchée, le chatbot déclare un manque d'informations, même si un modèle pourrait néanmoins convenir. Par ailleurs, le LLM peut occasionnellement halluciner des spécifications (stockage, connectivité...) non issues du contexte. La sensibilité au paramètre k , qui détermine le nombre de documents récupérés, et la capacité limitée des modèles légers peuvent aussi restreindre la précision ou la richesse de la compréhension.

Pour remédier à ces points et renforcer la fiabilité de ChatPhone, plusieurs axes d'amélioration s'ouvrent. Tout d'abord, l'exploitation plus fine des métadonnées notes moyennes, fourchettes de prix, dates de publication permettra de pré-filtrer automatiquement les extraits et d'améliorer la qualité du contexte avant génération. Ensuite, l'affinement du prompt via des exemples few-shot ciblés ou des instructions dynamiques selon le type de requête contribuera à réduire les digressions et à guider plus précisément la justification sur la caractéristique demandée. Par ailleurs, la montée en puissance du modèle de langage, en testant des LLM plus volumineux ou en procédant à un fine-tuning sur un corpus d'avis smartphone, offrira un meilleur compromis entre performance et efficacité.

De plus, l'instauration d'une boucle de feedback utilisateur, avec collecte systématique de notations de satisfaction, offrira un mécanisme de validation continue pour ajuster dynamiquement les paramètres de récupération (k , filtres) et de génération. Enfin, pour garantir la robustesse de l'évaluation, il conviendra d'étendre le protocole de test d'un jeu manuel de requêtes à un dispositif semi-automatisé ou crowdsourcé, apportant un volume et une diversité suffisants pour obtenir des métriques quantitatives fiables, tandis que l'enrichissement du corpus avec des sources complémentaires (blogs, forums spécialisés) et l'ajout de spécifications techniques détaillées renforcera la couverture des critères de sélection.

6.Conclusion

En somme, ChatPhone incarne une expérience ambitieuse de recommandation de smartphones, alliant la puissance d'une recherche augmentée (RAG) via FAISS à la finesse d'un modèle de langage pour synthétiser des extraits d'avis et délivrer une réponse ciblée. L'architecture mise en place, depuis l'ingestion des embeddings jusqu'à l'élaboration de prompts rigoureux, a démontré sa capacité à traiter des requêtes techniques variées tout en filtrant élégamment les questions hors domaine. Nos tests manuels ont confirmé la pertinence générale des recommandations, tout en soulignant la dépendance à la richesse du corpus d'avis, la sensibilité aux paramètres de récupération et le potentiel d'hallucinations inhérent aux modèles légers. Pour franchir un nouveau palier, il conviendra d'enrichir les métadonnées, de raffiner les prompts avec des exemples plus précis, d'instaurer une boucle de feedback utilisateur, de déployer des modèles plus performants et d'automatiser l'évaluation. Ces pistes ouvriront la voie à un assistant toujours plus fiable et pertinent, illustrant ainsi tout le potentiel évolutif de l'approche RAG et LLM pour révolutionner la recommandation produit.

7. Référence

[1] <https://www.ibm.com/fr-fr/think/topics/natural-language-processing>

[2] : <https://blog.gopenai.com/retrieval-augmented-generation-101-de05e5dc21ef>

[3] : <https://www.kaggle.com/datasets/grikomsn/amazon-cell-phones-reviews>